SABARAGAMUWA UNIVERSITY OF SRI LANKA
Faculty of Computing
Department of Data Science

Self-Supervised, Drift-Aware Zero-Day Intrusion Detection for IoT Networks

Cybersecurity Project Report
Project: iot-zeroday-ids

Submitted by
T. Kugashanth
Registration No: 22CDS0446
BSc (Hons) in Data Science

github.com/Kugashanth/iot-zeroday-ids
November 2026

Abstract

Intrusion detection systems based on signatures or supervised classifiers only recognise attack types that appear in their training data, so they miss the novel malware variants that IoT networks face. They also degrade when benign traffic changes over time (concept drift), producing floods of false alarms. This report presents **iot-zeroday-ids**, an open-source pipeline that addresses both problems. Five anomaly detectors — Isolation Forest, a scalable One-Class SVM, an Autoencoder, a Variational Autoencoder and a SCARF contrastive encoder — are trained on benign traffic only and evaluated under a strict zero-day protocol in which entire attack families (Mirai, Web, BruteForce and Spoofing) are hidden until the final test. The best detector is then wrapped in a label-free adaptation loop: ADWIN monitors trimmed anomaly scores, and on drift the model is fine-tuned on recent likely-benign traffic with a replay buffer and its threshold is re-calibrated using robust median/MAD statistics.

On a synthetic CIC-IoT2023-style dataset used to validate the pipeline, the VAE reached a zero-day ROC-AUC of 0.989 and detected 83.3% of unseen attacks at a 1% false-positive rate. Under concept drift, the static detector's false-positive rate rose to 16.8%, while the adaptive detector held it at 0.8%, at the cost of lower recall (0.67). The project is delivered as a tested Python package with a command-line interface, a Streamlit dashboard, a reproducible notebook, continuous integration and Docker support. Results on the real CIC-IoT2023 dataset are identified as the next step.

Keywords: intrusion detection, IoT security, zero-day attacks, anomaly detection, self-supervised learning, autoencoder, contrastive learning, concept drift, ADWIN

Table of Contents

Abstract....1
Table of Contents....2
1. Introduction....4
1.1 Background....4
1.2 Problem Statement....4
1.3 Objectives....4
1.4 Scope....4
1.5 Report Structure....5
2. Background and Related Work....6
2.1 Anomaly-Based Intrusion Detection....6
2.2 Autoencoders for Network Traffic....6
2.3 Self-Supervised Contrastive Learning for Tabular Data....6
2.4 Concept Drift Detection....6
2.5 The CIC-IoT2023 Dataset....6
3. System Architecture....7
4. Methodology....8
4.1 Data Loading and Family Mapping....8
4.2 Preprocessing....8
4.3 Zero-Day Split Protocol....8
4.4 Detectors....9
4.5 Thresholding....10
4.6 Evaluation Metrics....10
4.7 Drift Detection and Adaptation....10
5. Implementation....12
5.1 Technology Stack....12
5.2 Code Organisation....12
5.3 Command-Line Interface....12
5.4 Dashboard....13
5.5 Testing and Reproducibility....13
6. Experimental Setup and Results....14
6.1 Setup....14
6.2 Zero-Day Detection....14
6.3 Per-Family Analysis....15
6.4 Drift Adaptation....16
6.5 Computational Cost....17
7. Worked Example....19
7.1 Raw Flows....19
7.2 After Preprocessing....19
7.3 Scores and Decisions....19
7.4 Explaining the Alert....20
7.5 Scoring New Traffic....20
8. Discussion and Limitations....21
8.1 Discussion....21
8.2 Limitations....21
9. Conclusion and Future Work....22
9.1 Conclusion....22
9.2 Future Work....22
References....23

Appendix A: Default Configuration (Excerpt).....24
Appendix B: Repository Structure.....25

1. Introduction

1.1 Background

The Internet of Things connects cameras, sensors, smart plugs and industrial controllers that often run outdated firmware with weak default credentials. This makes them attractive targets: botnets such as Mirai recruit large numbers of IoT devices for distributed denial-of-service attacks, and new variants appear continuously. Network intrusion detection systems (NIDS) inspect traffic flows to detect such activity before it causes damage.

Two families of NIDS dominate. **Signature-based** systems match traffic against rules for known attacks. **Supervised machine-learning** systems learn to classify flows into labelled attack classes. Both share the same weakness: they can only recognise what they were shown. An attack family absent from the rules or the training labels — a zero-day attack — passes undetected.

1.2 Problem Statement

This project addresses two linked problems in IoT intrusion detection:

1. **Unknown attacks.** Detectors must flag attacks from families never seen during development, without any attack labels to learn from.
2. **Concept drift.** Benign traffic changes over time (firmware updates, new devices, new usage patterns). A detector calibrated on last month's traffic raises false alarms on this month's normal traffic, and security teams then ignore its alerts.

1.3 Objectives

1. Build a reproducible pipeline that trains anomaly detectors on benign IoT traffic only.
2. Evaluate detectors under a strict zero-day protocol in which whole attack families are held out.
3. Compare classical (Isolation Forest, One-Class SVM) and self-supervised deep detectors (Autoencoder, VAE, SCARF contrastive).
4. Design a label-free drift detection and adaptation loop, and measure its effect against a static detector.
5. Package the work as tested, documented, open-source software with a CLI and dashboard.

1.4 Scope

The project works with flow-level features (packet counts, rates, flag counts, sizes and timings) from the CIC-IoT2023 dataset schema. Payload inspection, encrypted-traffic analysis and response actions are out of scope. The pipeline was validated end to end on a synthetic dataset with the same schema; experiments on the real dataset are planned as future work (Chapter 9).

1.5 Report Structure

Chapter 2 reviews the relevant background. Chapter 3 describes the system architecture, and Chapter 4 details the methodology for each pipeline stage. Chapter 5 covers implementation, Chapter 6 the experimental setup and results, and Chapter 7 a worked example. Chapter 8 discusses findings and limitations, and Chapter 9 concludes.

2. Background and Related Work

2.1 Anomaly-Based Intrusion Detection

Anomaly-based detection models normal behaviour and flags deviations, so it can in principle detect attacks never seen before. The trade-off is false positives: unusual but benign traffic is also flagged. One-class methods such as Isolation Forest (Liu et al., 2008) and the One-Class SVM learn a description of normal data and score how far each new point falls outside it.

2.2 Autoencoders for Network Traffic

An autoencoder learns to compress and reconstruct its input. Trained only on benign flows, it reconstructs benign traffic well and attack traffic poorly, so reconstruction error serves as an anomaly score. Kitsune (Mirsky et al., 2018) showed that an ensemble of small autoencoders can detect network intrusions online. Variational autoencoders add a probabilistic latent space, which regularises what the model treats as normal.

2.3 Self-Supervised Contrastive Learning for Tabular Data

Contrastive learning trains an encoder to map two augmented views of the same sample close together and different samples apart. SCARF (Bahri et al., 2022) adapts this to tabular data by corrupting a random subset of features with values drawn from their empirical marginal distributions. The learned embeddings can be used for anomaly detection by measuring distance to known-normal embeddings.

2.4 Concept Drift Detection

Concept drift is a change in the data distribution over time. ADWIN (Bifet and Gavaldà, 2007) maintains a variable-length window over a data stream and signals drift when the means of two sub-windows differ by more than a statistical bound allows. Page-Hinkley is a cumulative-sum alternative. In intrusion detection, drift adaptation is risky because an attacker could slowly shift the model's notion of normal; any adaptation scheme must limit this.

2.5 The CIC-IoT2023 Dataset

CIC-IoT2023 (Neto et al., 2023) was captured from a testbed of 105 IoT devices and contains 33 attacks in seven categories plus benign traffic, released as CSV files of flow features. Its size (about 46 million flows across 169 files) and breadth of attack families make it suitable for zero-day evaluation: whole families can be held out while enough data remains for training.

3. System Architecture

The system is organised as a seven-stage pipeline (Figure 3.1). Stages 1–6 build and evaluate the detectors and are run by `iotids train`. Stage 7 deploys a trained detector on a stream of traffic and keeps it accurate as benign traffic changes; it is run by `iotids stream`.

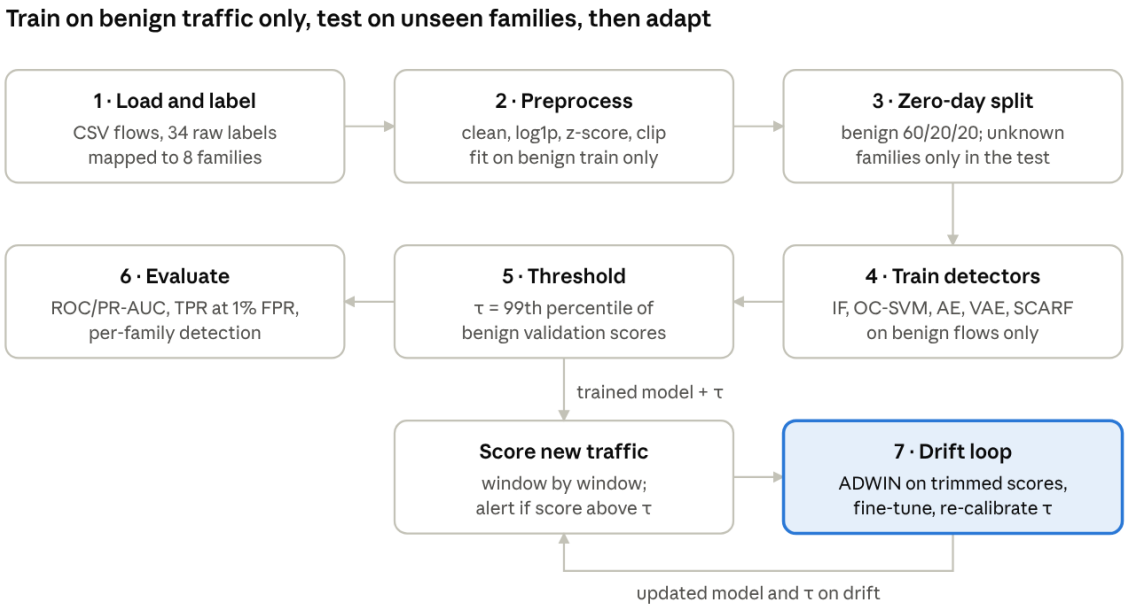


Figure 3.1: End-to-end pipeline of iot-zeroday-ids: benchmark stages 1–6 and the deployment drift loop (stage 7)

Table 3.1: Pipeline stages, modules and data flow

Stage	Module	Input → Output
1. Load and label	`data.py`	CSV files → table with label and family columns
2. Preprocess	`data.py`	Raw features → scaled float32 matrix (fit on benign train)
3. Zero-day split	`data.py`	Table → train / validation / test / known sets
4. Train detectors	`models/`	Benign train (+ benign validation) → five fitted models
5. Threshold	`metrics.py`	Benign validation scores → threshold τ per model
6. Evaluate	`metrics.py`, `plots.py`	Test scores → metrics, CSV files, figures
7. Drift loop	`drift.py`	Stream of flows → alerts + self-updated model and τ

All detectors share one interface (`fit`, `score`, `adapt`, `save`, `load`) and output an anomaly score where higher means more anomalous. This makes thresholding, evaluation and drift handling independent of the model used, and new detectors can be added by implementing the same five methods.

4. Methodology

4.1 Data Loading and Family Mapping

Each CIC-IoT2023 row is one network flow with 46 numeric features (for example `flow_duration`, `Rate`, `syn_flag_number`, `Tot size` and `IAT`) and a label. The loader reads a configurable number of files, samples a fraction of each to bound memory, and maps the raw labels into eight families (Table 4.1). Because DDoS and DoS dominate the raw data, rows per family are capped.

Table 4.1: Mapping of CIC-IoT2023 labels to attack families

Family	Example raw labels
Benign	BenignTraffic
DDoS	DDoS-ICMP_Flood, DDoS-SYN_Flood, DDoS-UDP_Flood
DoS	DoS-TCP_Flood, DoS-HTTP_Flood
Mirai	Mirai-greeth_flood, Mirai-udpplain
Recon	Recon-PortScan, Recon-OSScan, VulnerabilityScan
Spoofing	MITM-ArpSpoofing, DNS_Spoofing
BruteForce	DictionaryBruteForce
Web	SqlInjection, XSS, CommandInjection, Backdoor_Malware, Uploading_Attack, BrowserHijacking

4.2 Preprocessing

Preprocessing is fit on the **benign training rows only** and then applied unchanged to every other split, so no information about attacks influences scaling. The steps are: (i) keep numeric columns and drop labels, timestamps and identifiers; (ii) replace infinite values with NaN and impute the benign median; (iii) drop columns that are constant in benign traffic; (iv) apply log1p to heavy-tailed non-negative columns (skewness above 1); and (v) standardise with benign statistics and clip to ±10 (Equation 1).

$$z_j = \mathrm{clip}\left(\frac{\log(1+x_j) - \mu_j^{\text{benign}}}{\sigma_j^{\text{benign}}},\ -10,\ 10\right) \tag{1}$$

4.3 Zero-Day Split Protocol

Benign rows are split 60/20/20 into training, validation and test sets. Attack families listed as *unknown* (default: Mirai, Web, BruteForce, Spoofing) appear only in the test set. The remaining *known* families (DDoS, DoS, Recon) form a separate evaluation set; models are never trained on them either, but they allow a comparison between attacks similar to familiar ones and truly novel ones. Holding out complete families prevents the model from learning each attack's fingerprint, which a random split of attack data would allow.

Table 4.2: Composition of the splits in the demonstration run

Split	Contents	Purpose	Rows
Train	Benign only	Fit preprocessor and models	36,000
Validation	Benign only	Early stopping, threshold	12,000
Test	Benign + unknown families	Zero-day evaluation	12,000 + 12,000
Known	Known families only	Secondary evaluation	12,000

4.4 Detectors

4.4.1 Isolation Forest

Isolation Forest builds 300 random trees on subsamples of 1,024 benign flows. Anomalous points are isolated in fewer random splits; the score is the negative average path length.

4.4.2 One-Class SVM

The exact One-Class SVM scales quadratically with sample size. The implementation approximates an RBF kernel with 300 Nyström components and fits a linear One-Class SVM with stochastic gradient descent (ν = 0.05) on up to 50,000 flows. The score is the negative decision function.

4.4.3 Autoencoder

The autoencoder is a mirrored multilayer perceptron (input → 128 → 64 → 16 → 64 → 128 → input) with batch normalisation, ReLU and dropout of 0.1. It is trained with AdamW, a cosine learning-rate schedule, gradient clipping and early stopping on the benign validation loss. The anomaly score is the mean squared reconstruction error over the d features (Equation 2).

$$s_{\mathrm{AE}}(x) = \frac{1}{d}\sum_{j=1}^{d}(x_j - \hat{x}_j)^2 \tag{2}$$

4.4.4 Variational Autoencoder

The VAE uses the same body with a 16-dimensional Gaussian latent space. Its loss adds a β-weighted Kullback-Leibler term (β = 0.5) to the reconstruction error (Equation 3). At scoring time the posterior mean is decoded and the reconstruction error is used as the score.

$$\mathcal{L}_{\mathrm{VAE}} = \frac{1}{d}\left(\|x - \hat{x}\|^2 + \beta\,\mathrm{KL}\left(q(z \mid x)\,\|\,\mathcal{N}(0, I)\right)\right) \tag{3}$$

4.4.5 SCARF Contrastive Encoder

For each flow, two corrupted views are created by replacing a random 30% of features with values taken from other flows in the batch, plus small Gaussian noise. An encoder (input → 256 → 128 → 64) and projection head are trained with the NT-Xent loss at temperature T = 0.5 (Equation 4), which pulls the two views of the same flow together.

$$\ell_i = -\log \frac{\exp(\mathrm{sim}(z_i, z_i^+)/T)}{\sum_{k \neq i} \exp(\mathrm{sim}(z_i, z_k)/T)} \tag{4}$$

After training, the embeddings of up to 20,000 benign flows form a memory bank. A new flow's score is its mean distance to the k = 5 nearest benign embeddings in L2-normalised space (Equation 5).

$$s_{\mathrm{SCARF}}(x) = \frac{1}{k} \sum_{m=1}^{k} \| e(x) - e_{(m)} \|_2 \tag{5}$$

4.5 Thresholding

Each model scores the benign validation set, and the threshold τ is the 99th percentile of those scores (Equation 6). By construction about 1% of benign flows exceed τ, so the expected false-positive rate is about 1%. No attack labels are used.

$$\tau = Q_{0.99}\big(\{ s(x) : x \in \text{benign validation} \}\big), \qquad \text{alert} \Leftrightarrow s(x) > \tau \tag{6}$$

4.6 Evaluation Metrics

Table 4.3: Evaluation metrics

Metric	Question answered
ROC-AUC	How well scores rank attacks above benign flows overall
PR-AUC	Ranking quality when attacks are the minority class
TPR @ 1% FPR	Share of attacks caught when 1% of benign flows may raise alerts
FPR @ 95% TPR	False-alarm rate required to catch 95% of attacks
Precision, recall, F1, FPR	Alert quality at the chosen threshold τ
Per-family detection rate and ROC-AUC	Which attack families are missed
Fit time, µs per flow	Training cost and inference latency

TPR at 1% FPR is treated as the primary operational metric, because high false-positive rates cause alert fatigue and make an IDS unusable in practice.

4.7 Drift Detection and Adaptation

In deployment, flows arrive in windows of w flows (1,000 in the demonstration). For each window the loop shown in Figure 4.1 runs. Labels are used only to compute reported metrics, never by the loop.

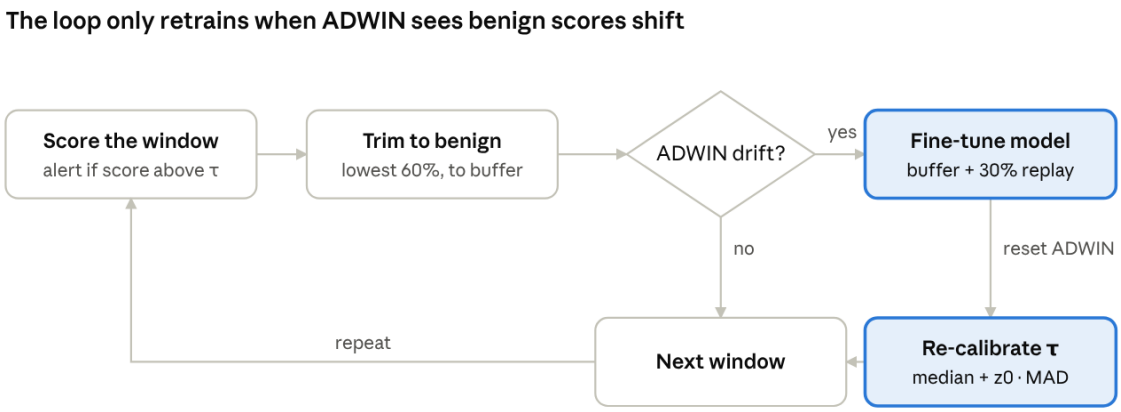


Figure 4.1: The label-free drift detection and adaptation loop executed once per window

1. **Score and alert:** every flow is scored and flagged if its score exceeds τ.
2. **Trim:** flows scoring at or below the window's 60th percentile are taken as likely benign. This is robust while attacks are a minority of the window.
3. **Detect:** the trimmed scores are fed to ADWIN (δ = 0.002); the likely-benign rows are appended to a FIFO buffer of recent flows.
4. **Adapt:** when ADWIN signals drift, at least five windows have passed since the last adaptation and the buffer is large enough, the model is fine-tuned on the buffer mixed with a 30% replay sample of the original benign training data, at 0.3× the original learning rate. The replay prevents catastrophic forgetting.
5. **Re-calibrate:** the threshold is reset to keep its original standardised position, using the median and MAD of recent scores (Equation 7), which tolerate up to 50% contamination by attacks. ADWIN is then reset.

$$z_0 = \frac{\tau_0 - \mathrm{med}_0}{\sigma_0}, \quad \tau_{\mathrm{new}} = \mathrm{med}_{\mathrm{recent}} + z_0\,\sigma_{\mathrm{recent}}, \quad \sigma = 1.4826 \cdot \mathrm{MAD} \tag{7}$$

Where the data has timestamps, drift is natural. The CIC-IoT2023 CSV files have none, so for that dataset the stream experiment injects a simulated drift: a shift of 1.5 standard deviations on eight random features from 40% of the way through the stream, imitating a network-wide change such as a firmware update.

5. Implementation

5.1 Technology Stack

Table 5.1: Technology stack

Area	Tools
Language and packaging	Python 3.10+, setuptools (`pyproject.toml`), installable as `iotids`
Machine learning	PyTorch 2 (AE, VAE, SCARF), scikit-learn (Isolation Forest, Nyström + SGD One-Class SVM)
Stream learning	river (ADWIN, Page-Hinkley)
Data and plots	pandas, NumPy, Matplotlib, Plotly
Interface	argparse CLI, Streamlit dashboard, Jupyter notebook
Quality	pytest (27 tests), ruff, GitHub Actions CI, Docker

5.2 Code Organisation

Table 5.2: Package modules

Module	Responsibility
`config.py`	YAML configuration with inheritance; random seeds
`synthetic.py`	Synthetic CIC-IoT2023-like data generator with drift
`data.py`	Loading, family mapping, Preprocessor, zero-day splits
`models/`	Detector interface; classical and deep detectors
`metrics.py`	Thresholds, ROC/PR metrics, per-family metrics, robust statistics
`drift.py`	Streaming runner, ADWIN adaptation, static vs adaptive comparison
`pipeline.py`	`run_batch` (benchmark) and `run_stream` (drift experiment)
`plots.py`	Report figures
`cli.py`	Command-line entry point

5.3 Command-Line Interface

```
pip install -e ".[app,dev]"
iotids synth   --out data/synthetic
iotids train   --config configs/synthetic.yaml
iotids stream  --config configs/synthetic.yaml --model ae
iotids all     --config configs/default.yaml
iotids predict --run runs/<run> --model vae --input new.csv --output scored.csv
streamlit run app/streamlit_app.py
```

Every run writes a timestamped folder containing the exact configuration, split composition, metrics (CSV and JSON), per-family results, all test scores, the saved preprocessor, models and thresholds, and figures. This makes every reported number traceable to the configuration that produced it.

5.4 Dashboard

The Streamlit dashboard reads the run folders and provides four views: the zero-day benchmark (metrics, ROC curves and per-family heatmaps); a threshold explorer in which the benign quantile can be adjusted while recall and false-positive rate update live; a drift view comparing static and adaptive detectors window by window; and a CSV upload page that scores new flows with any saved model.

5.5 Testing and Reproducibility

The 27 automated tests cover label-to-family mapping, the absence of benign leakage between splits, preprocessing of infinite values, that every detector separates attacks (ROC-AUC above 0.75) and survives a save/load round trip, that robust statistics resist 25% contamination, that adaptation lowers false-positive rate under synthetic drift, and an end-to-end run including prediction. Tests run on every push via GitHub Actions on Python 3.10 and 3.12. Fixed seeds make runs repeatable.

6. Experimental Setup and Results

6.1 Setup

All experiments reported here use the synthetic dataset produced by `iotids synth`: 60,000 benign flows from three device profiles and 1,500 flows for each of 16 attack labels (84,000 flows in total), with 24 CIC-IoT2023-style features and a benign distribution shift at ts = 0.6. The synthetic data validates that the pipeline works; its absolute numbers are not evidence of performance on real traffic. Training ran on a laptop-class CPU with the default configuration (threshold quantile 0.99, seed 42).

6.2 Zero-Day Detection

Table 6.1: Zero-day results on unknown families (12,000 benign + 12,000 attack flows)

Model	ROC-AUC	PR-AUC	TPR@1% FPR	FPR@95% TPR	Precision	Recall	F1
VAE	**0.989**	**0.989**	**0.833**	**0.058**	0.989	**0.816**	**0.894**
Autoencoder	0.987	0.988	0.815	0.073	0.989	0.798	0.884
SCARF contrastive	0.979	0.980	0.734	0.112	0.987	0.728	0.838
One-Class SVM	0.956	0.961	0.584	0.283	0.983	0.576	0.726
Isolation Forest	0.930	0.931	0.423	0.327	0.977	0.415	0.582

The three self-supervised deep models clearly outperform the classical baselines. The VAE is best on every ranking metric and catches 83.3% of unseen attacks at a 1% false-positive rate, against 42.3% for Isolation Forest. All models keep their false-positive rate at 0.9–1.0%, confirming that the benign-quantile threshold behaves as designed. Figure 6.1 shows the ROC curves on a logarithmic false-positive axis, where the differences at low alarm rates are clearest.

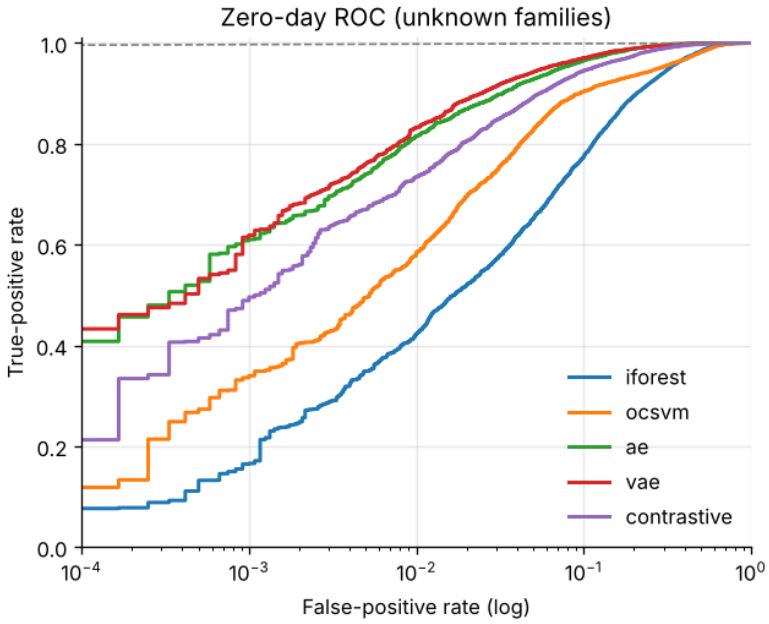


Figure 6.1: Zero-day ROC curves (log-scale false-positive rate)

6.3 Per-Family Analysis

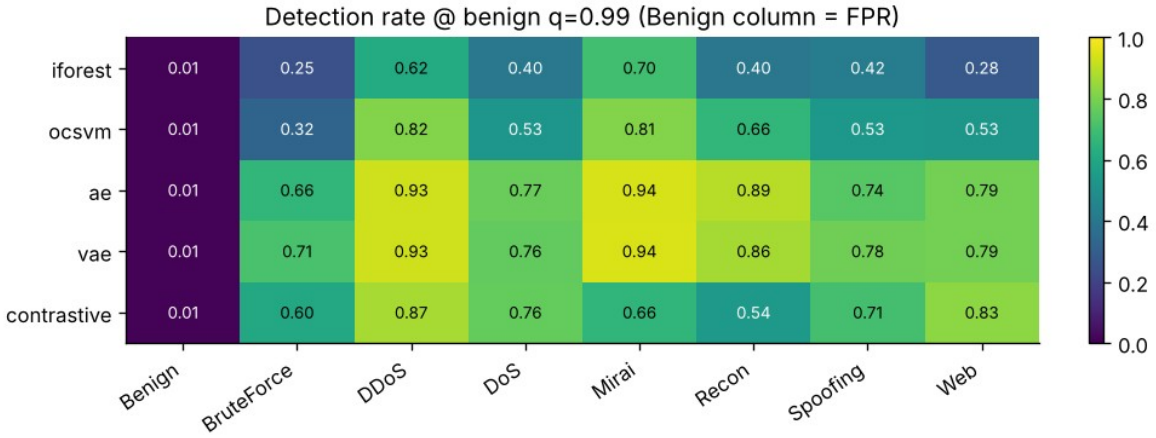


Figure 6.2: Detection rate per family at the 0.99 benign threshold (the Benign column is the false-positive rate)

Loud, volumetric attacks (Mirai, DDoS) are detected at 93–94% by the autoencoders. Subtle families are harder: BruteForce is the weakest for every model (66–71% for the autoencoders) and Spoofing and Web follow. The contrastive model is the best on Web attacks (83.1%) but weaker on Mirai (65.7%) and Recon (54.0%), which suggests that combining detectors could help. Known families score only slightly higher than unknown ones (VAE ROC-AUC 0.991 versus 0.989), as expected for models that never see any attack.

Table 6.2: Known-family results (DDoS, DoS, Recon), never used for training

Model	ROC-AUC	TPR@1%FPR	Recall	F1
Autoencoder	0.993	0.890	0.876	0.929
VAE	0.991	0.876	0.862	0.922
SCARF contrastive	0.982	0.726	0.719	0.832
One-Class SVM	0.968	0.696	0.686	0.809
Isolation Forest	0.952	0.491	0.480	0.644

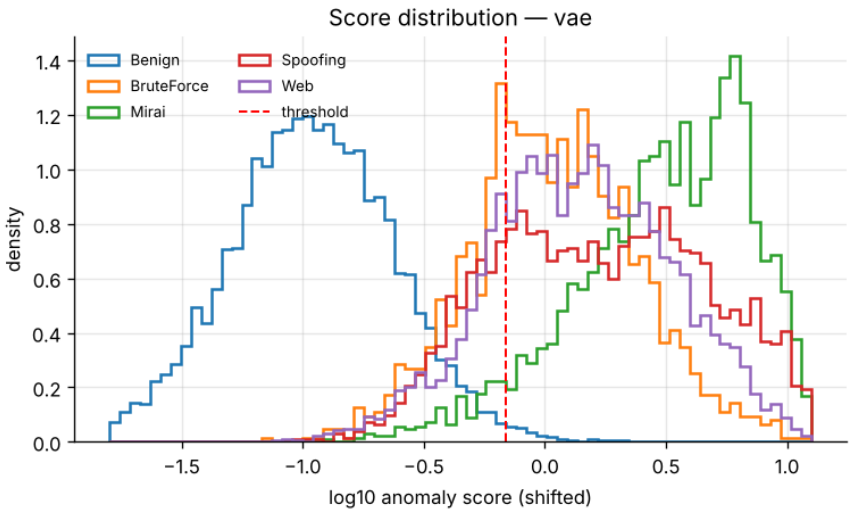


Figure 6.3: VAE anomaly score distribution by family, with the decision threshold

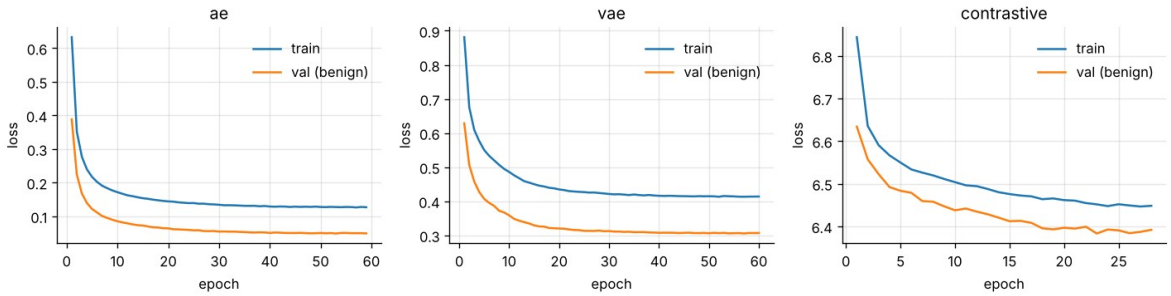


Figure 6.4: Training and benign validation loss for the deep models (early stopping on validation loss)

6.4 Drift Adaptation

For the stream experiment the autoencoder was trained on the earliest 40% of benign traffic, calibrated on the next 10%, and then run over the remaining stream of 42 windows of 1,000 flows with attacks mixed in. The benign shift begins in window 8.

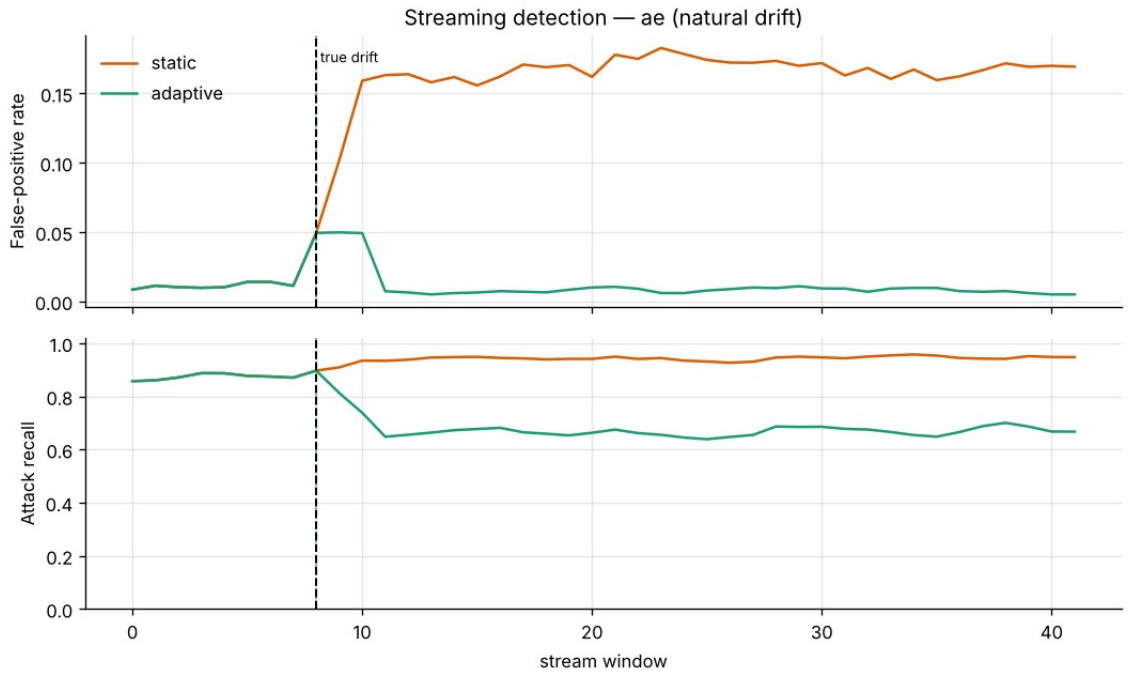


Figure 6.5: False-positive rate and attack recall per window for the static and adaptive detectors

Table 6.3: Stream results before and after drift (autoencoder)

Detector	Period	FPR	Recall	Windows
Static	Pre-drift	2.5%	0.88	9
Static	Post-drift	16.8%	0.94	33
Adaptive	Pre-drift	2.5%	0.88	9
Adaptive	Post-drift	0.8%	0.67	33

ADWIN signalled drift in window 8, the window in which it began, and one adaptation (threshold 0.356 → 0.994) was sufficient for the rest of the stream. Without adaptation, one benign flow in six raised an alert. The static model's higher post-drift recall is not a benefit: it results from flagging almost everything. Adaptation restores a usable alarm rate but lowers recall from 0.88 to 0.67, discussed in Chapter 8.

6.5 Computational Cost

Table 6.4: Training time and inference latency

Model	Fit time (s)	Inference (µs per flow)
Isolation Forest	0.6	17.1
One-Class SVM	0.3	3.3
Autoencoder	27	3.3
VAE	27	3.3
SCARF contrastive	44	67

All models score flows in microseconds, fast enough for real-time use. SCARF is slowest at inference because of its k-nearest-neighbour search over the memory bank.

7. Worked Example

This chapter follows three flows through the trained pipeline: one benign flow, one Mirai flow and one XSS (Web family) flow. Mirai and Web are both unknown families, so neither the models nor the thresholds had seen any example of them.

7.1 Raw Flows

Table 7.1: Selected raw feature values of the three flows

Flow	Label	Tot size	AVG	Header_Length	Std
A	BenignTraffic	5.08	3.96	12.03	8.79
B	Mirai-greeth_flood	54.99	42.33	405.05	8.61
C	XSS	87.60	38.24	11.13	55.70

7.2 After Preprocessing

Table 7.2: Standardised values (benign statistics; near 0 = typical)

Flow	Tot size	AVG	Header_Length	Std
A Benign	−0.50	−0.59	−0.32	−1.57
B Mirai	0.23	0.73	**10.00** (clipped)	−1.60
C XSS	0.70	0.59	−0.35	0.51

The Mirai flow's header length lies more than ten standard deviations above benign traffic. The XSS flow has no single extreme feature; it differs in how several features combine, which makes it harder to detect.

7.3 Scores and Decisions

Table 7.3: Anomaly scores against each model's threshold

Model	τ	A Benign	B Mirai	C XSS
Isolation Forest	0.511	0.346 benign	0.435 benign	0.467 benign
One-Class SVM	0.0045	−0.0034 benign	0.0039 benign	0.0051 **alert**
Autoencoder	0.278	0.010 benign	0.475 **alert**	0.820 **alert**
VAE	0.700	0.028 benign	0.968 **alert**	1.269 **alert**
SCARF contrastive	0.311	0.092 benign	0.481 **alert**	0.516 **alert**

All three deep models flag both unknown attacks and pass the benign flow. Isolation Forest misses both and the One-Class SVM catches only the XSS flow, narrowly — consistent with the benchmark results in Chapter 6.

7.4 Explaining the Alert

The features that the autoencoder reconstructed worst indicate why each alert was raised, and are the first thing an analyst would examine:

Table 7.4: Top three per-feature reconstruction errors (autoencoder)

Flow	Largest squared errors
A Benign	Duration 0.06, Header_Length 0.03, IAT 0.02
B Mirai	Header_Length 2.64, Number 1.39, urg_count 0.99
C XSS	fin_count 7.49, syn_count 3.43, fin_flag_number 1.26

7.5 Scoring New Traffic

```
iotids predict --run runs/synthetic-batch-20261109-232413 --model vae \
               --input new_flows.csv --output scored.csv
```

The output file contains the input columns plus `anomaly_score` and `is_attack_pred` (1 = alert). The same function is available through the dashboard's upload page.

8. Discussion and Limitations

8.1 Discussion

The results support the central idea of the project: models trained only on benign traffic can detect attack families they have never seen, at an operationally useful false-alarm rate. Self-supervised deep models capture relationships between features rather than individual feature ranges, which explains their advantage over Isolation Forest on subtle families. The drift experiment shows that a static detector quickly becomes unusable when benign traffic shifts, and that a label-free loop can restore its false-positive rate within one window.

8.2 Limitations

- **Synthetic data.** The generator produces cleaner separation than real traffic; the absolute numbers will change on CIC-IoT2023 and must not be cited as real-world performance.
- **Simulated drift on CIC-IoT2023.** Its CSV release has no timestamps, so drift on that dataset must be simulated.
- **Adaptation poisoning.** An attacker who increases activity slowly could be absorbed into the model's notion of normal. Trimming, replay, cooldown and the fixed standardised threshold limit this risk but do not eliminate it; production use should require human approval of adaptations.
- **Recall after adaptation.** Recall fell from 0.88 to 0.67 after adaptation because the model becomes tuned to the new normal, making nearby attacks harder to separate. The threshold quantile, trim quantile and replay ratio control this trade-off.
- **Flow features only.** Payload content and encrypted traffic are not analysed.

9. Conclusion and Future Work

9.1 Conclusion

This project built and validated iot-zeroday-ids, a reproducible pipeline for zero-day intrusion detection in IoT networks. Five detectors trained on benign traffic only were compared under a strict protocol that hides whole attack families until testing. The VAE performed best, detecting 83.3% of unseen attacks at a 1% false-positive rate on synthetic data. A label-free ADWIN-based adaptation loop reduced post-drift false alarms from 16.8% to 0.8%. The work is packaged as tested open-source software with a CLI, dashboard, notebook, CI and Docker support.

9.2 Future Work

1. Run the full benchmark on CIC-IoT2023, TON_IoT and N-BaIoT and report real results.
2. Perform leave-one-family-out evaluation across all seven attack families.
3. Add per-feature and SHAP explanations of alerts to the dashboard.
4. Explore ensembles that combine the autoencoder and contrastive detectors.
5. Investigate federated training across devices (Flower) and ONNX export for edge inference.
6. Evaluate adversarial robustness and defences against adaptation poisoning.

References

Bahri, D., Jiang, H., Tay, Y. and Metzler, D. (2022) 'SCARF: Self-Supervised Contrastive Learning using Random Feature Corruption', *International Conference on Learning Representations (ICLR)*.

Bifet, A. and Gavaldà, R. (2007) 'Learning from Time-Changing Data with Adaptive Windowing', *Proceedings of the SIAM International Conference on Data Mining (SDM)*, pp. 443–448.

Kingma, D.P. and Welling, M. (2014) 'Auto-Encoding Variational Bayes', *International Conference on Learning Representations (ICLR)*.

Liu, F.T., Ting, K.M. and Zhou, Z.-H. (2008) 'Isolation Forest', *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pp. 413–422.

Mirsky, Y., Doitshman, T., Elovici, Y. and Shabtai, A. (2018) 'Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection', *Network and Distributed System Security Symposium (NDSS)*.

Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R. and Ghorbani, A.A. (2023) 'CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment', *Sensors*, 23(13), 5941.

Schölkopf, B., Platt, J.C., Shawe-Taylor, J., Smola, A.J. and Williamson, R.C. (2001) 'Estimating the Support of a High-Dimensional Distribution', *Neural Computation*, 13(7), pp. 1443–1471.

Appendix A: Default Configuration (Excerpt)

```
data:
  path: data/CICIoT2023
  label_col: label
  max_files: 20
  sample_frac: 0.25
  max_rows_per_family: 200000
split:
  train_frac: 0.6
  val_frac: 0.2
  unknown_families: [Mirai, Web, BruteForce, Spoofing]
threshold:
  quantile: 0.99
models:
  - name: ae
    params: {hidden: [128, 64], latent: 16, dropout: 0.1, epochs: 60, patience:
6}
  - name: vae
    params: {hidden: [128, 64], latent: 16, beta: 0.5, epochs: 60}
  - name: contrastive
    params: {hidden: [256, 128], latent: 64, corruption: 0.3, temperature: 0.5,
k: 5}
stream:
  window: 2000
  trim_quantile: 0.6
  detector: adwin
  delta: 0.002
  replay_ratio: 0.3
  cooldown_windows: 5
```

Appendix B: Repository Structure

```
iot-zeroday-ids/
├── configs/          default.yaml (CIC-IoT2023), synthetic.yaml (demo)
├── src/iotids/       package: data, models/, metrics, drift, pipeline, cli
├── app/              Streamlit dashboard
├── notebooks/        01_quickstart.ipynb
├── tests/            27 pytest tests
├── docs/images/      README figures
├── Dockerfile, docker-compose.yml, Makefile
└── .github/workflows/ci.yml
```